越肩 / 第一人称镜头

把一款等距俯视的 Switch 游戏，在图形翻译层改成「射击游戏式」追随视角的技术方案与实现

D2R Mobile · 实验功能，缺省关；环境变量 `DM_CAM=1` 打开越肩、`DM_CAM=2` 第一人称；只在 Metal 后端（iOS / Mac）上有效

> **一句话：**游戏本身一行没改。画面这头，在 GPU 指令翻译到 Metal 的那一层，把游戏每帧上传的「主相机」常量缓冲换成我们自己算的透视矩阵；输入那头，把玩家在透视画面上的触点和摇杆方向，反算回原版正交画面上的点和方向再喂给游戏。游戏以为自己还是俯视，玩家看到的是越肩追随视角。

1. 背景：为什么能做

- **原版镜头**是固定朝向的等距正交投影（视线下俯 30°、水平转 45°、平行投影、没有近大远小、没有天空）。游戏的命中判定、点击、摇杆方向都建立在这个画面上。
- **我们的位置**：项目在 iOS 上直接跑 Switch 版程序，图形 API（NVN）由我们逐条翻译成 Metal。游戏每次绘制前绑定的常量缓冲（矩阵等参数）都要经过我们这一层，所以可以在 GPU 读到它之前换掉内容。
- **关键观察**：游戏 3D 场景本来就是真 3D 模型（地形、墙、角色都有高度），只是被正交相机「拍扁」了。换一个透视相机，整个世界就立起来了；剩下的问题是游戏不知道相机变了。

2. 总体结构

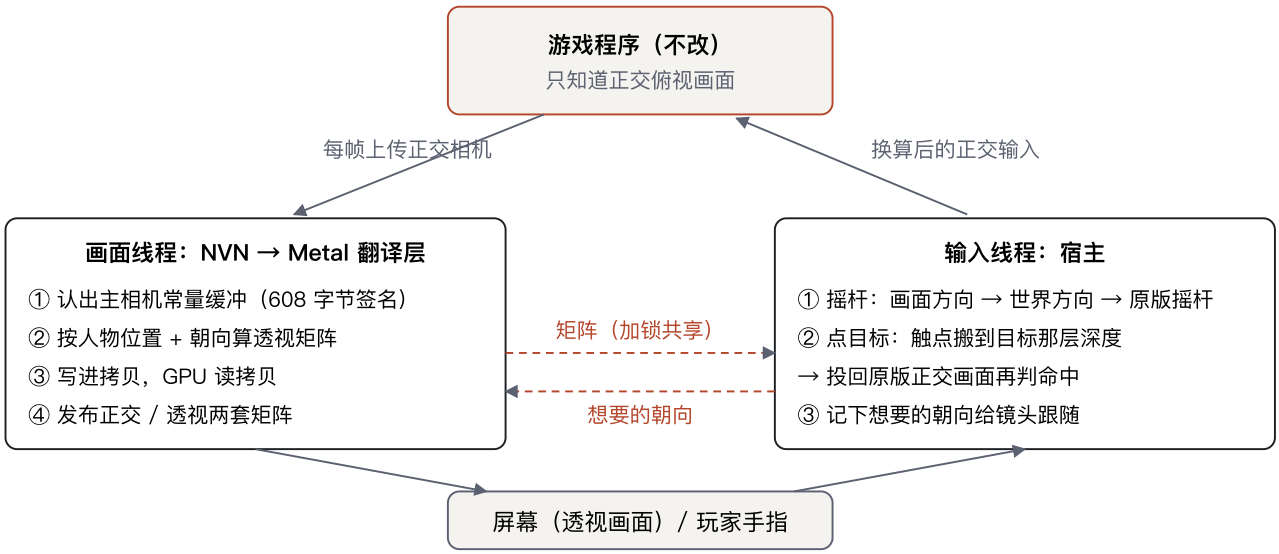


图 1　两半：画面线程换相机，输入线程反算输入；两边只通过两份矩阵和一个朝向交换信息

代码位置：渲染侧在 `dm/gfx/nvn/nvn_mtl.m`（`cam_sig / cam_build / cam_patch`），输入侧在 `dm/host/dm_cam.c`，另在取手柄处（`hle_hid.c`）、点目标处（`dm_world.c`）各接一处。**缺省关；关着时这些路径原样返回，不改任何值**（最初实现实测关着时 0 像素差、0 行镜头日志）。

3. 渲染侧：把正交相机换成透视相机

3.1 认出「主相机」缓冲

游戏每帧会上传很多常量缓冲，阴影图那几份也带相机矩阵（但是光源视角）。我们只认主相机，用内容签名识别：

条件	含义
大小 ⩾ 608 字节	主相机常量块的布局长度
偏移 +80 视图矩阵第 1 行 ≈ (0.7071, −0.3536, 0.6124)，第 2 行 ≈ (0, 0.866, 0.5)	原版固定的等距朝向（俯 30° / 转 45°）
偏移 +208 投影矩阵第 3 列为 0、右下角 = 1	正交投影

命中后不改原缓冲，而是拷一份到我们自己的环形缓冲（4 MB）里改，绑定时把 GPU 地址指向拷贝。同一帧同一地址只改一次，复用结果。

3.2 要重写的东西

偏移	内容	处理
+80 / +144	视图矩阵 / 逆视图	按新镜头位置重算
+208 / +272	投影 / 逆投影	换成透视（反向 Z，见下）
+336 / +400	视图投影 / 逆视图投影	重算
+464	上一帧视图投影	填我们**上一帧改过的**那份，否则运动矢量 / 时域抗锯齿会糊
+592	深度还原参数（远、近、?、正交标志）	标志置 0 走着色器里的透视分支，xy 填 (B, −A)，深度 d → 距离 = B / (d + A)
片元槽 10（1440 字节场景常量）	+16 镜头位置、+32 视线方向、+848 线性深度起点	同步改；起点原版是 550（正交镜头离地很远），透视改成 0

有两处**故意不改**：

- **阴影图**的相机（光源视角，不是签名里的朝向，本来也不会命中）。
- **一个按正交视锥做剔除的计算着色器**（按着色器指纹排除）。改了它整片地面和墙体消失；不改它，游戏照旧按原版画面范围挑要画的东西。

另外开镜头时关掉我们自己的「运行时外推」（一种省电的插帧优化），它假设相机是原版的。

3.3 投影矩阵（反向 Z）

游戏用的是反向 Z 深度（近 = 1、远 = 0），透视投影照这个约定构造。行向量约定（裁剪坐标 = [x y z 1] × M）：

```
sx = 1 / tan(水平视角 / 2)        sy = sx × 原投影的纵横比 (pr[5] / pr[0])
A  = n / (F − n)                  B  = n·F / (F − n)        n = 0.3, F = 3000
P  = | sx  0   0   0 |
     | 0   sy  0   0 |
     | 0   0   A  −1 |
     | 0   0   B   0 |            视图 z = −n → 深度 1, z = −F → 深度 0
```

视图矩阵用常规 look-at：后 = 归一化(眼 − 看点)，右 = 上(0,1,0) × 后，上 = 后 × 右。逆矩阵用 4×4 高斯消元（双精度）。

3.4 镜头放在哪

人物位置不用读游戏内存：原版正交镜头总是对准人物，所以取原版逆视图里的镜头位置和视线方向，求视线与地面 y = 0 的交点，就是人物脚下。

参数	越肩（已做成可玩模式）	第一人称（只做了演示）
镜头位置	人物身后 20、高 13（受俯仰限制可再抬高）	人物前方 2.2、高 6（眼位）
看向	人物前方 4、高 3	正前方，下俯 8°
水平视角	65°	75°

所有参数都留了环境变量（`DM_CAM_DIST / H / LOOK / LOOKH / FOV / EYE / FWD / PITCH / TAU / NEAR`），调手感不用重编。

3.5 镜头跟随

- **想要的朝向**按优先级取：瞄准拖拽方向 > 移动摇杆方向（都由输入侧换成世界方向）> 人物逐帧位移方向；都没有就保持不动。
- **转法**：按墙钟做一阶阻尼转角度，`角度 += 差角 × (1 − e^(−dt/τ))`，τ = 0.35 秒，走最短方向。按墙钟而不是按帧，30 / 40 / 60 帧下转速一致。
- **重置**：首帧，或人物一帧位移超过 20（换区域、传送、过门），朝向重置为原版视线的水平方向 (−0.707, −0.707)。这样一进图画面和原版同朝向，玩家不迷路。

3.6 俯仰限制：不让地平线进画面

原版是俯视游戏，**根本没有天空**。透视镜头一抬头，地平线以上全是黑的。约束画面上沿至少在水平线下 3°；竖直半视角按本帧纵横比算（iPad 比 iPhone 方，同样水平视角下竖直视角更大），镜头不够高就自动抬高：

```
tan(竖直半角) = tan(水平视角/2) × 高/宽
最低镜头高度 = 看点高 + (距离 + 前看) × tan(竖直半角 + 3°)
```

4. 输入侧：游戏只认原版画面

画面是透视的，但游戏的点击、命中、摇杆全按原版正交画面算。我们不碰游戏，只在输入喂给游戏前换算一次。画面线程每帧把**原版正交视图投影（及逆）、透视视图投影（及逆）、人物落地点**加锁发布出来，30 帧内

没更新就视为镜头关闭、输入原样放行。

4.1 基本换算：透视画面点 → 原版画面点

1. 屏幕点 → 裁剪坐标，用透视逆矩阵在近、远两个深度各反投影一个点，得到一条射线。
2. 射线与某个平面求交，得到世界坐标。
3. 世界坐标用原版正交视图投影投回去，得到「原版画面上的同一个东西」。

4.2 摇杆：相对镜头

原版里推摇杆的方向，对应画面上 (x, y/2) 的方向（等距 2:1）。越肩模式要的是「推上 = 朝镜头正前方走」：

1. 摇杆 → 画面方向 (x, y/2)，归一化。
2. 取镜头的水平朝向 f（屏幕中心射线的 xz），画面方向按「上 = f、右 = (−f.z, f.x)」转成世界方向。
3. 从人物脚下沿世界方向取两点，用原版正交矩阵投回原版画面，得到原版画面方向，再按 2:1 反换回摇杆值，幅度保持不变。

注意：不要用「透视画面上人物所在的点」去推方向，那样随人物在画面里的位置会偏几度（实测推正上偏 6°）；直接用镜头水平朝向转角度最准。

> **坑：按住右推原地转圈。**摇杆相对镜头，镜头又追摇杆的世界方向，两者互相追，按住右推会绕圈跑（实测 30 帧转半圈）。
> **改法：参照镜头按「这一推」锁住**——手指按下（摇杆从 0 变非 0）那一刻的镜头当参照，松手前一直用它。效果：按住右推 = 朝按下时画面的右边一直直线跑，镜头约 1.5 秒转到人背后；下一次推再按新镜头算。左右两根摇杆各锁各的。

4.3 点目标：搬到目标那一层深度

直接把触点射线交地面不行：人和怪物有身高，点在身上的射线会落到他身后的地面。做法是**逐个候选目标**换：

1. 把该目标原版画面上的脚底点还原到世界（正交视线平行，两个深度连线交 y = 0）。
2. 取过脚底、法向 = 射线水平方向的竖直平面，触点射线与它求交。交点在镜头背后（t ⩽ 0）的目标直接不算。
3. 交点投回原版画面，再用原有的那套身体框 / 占地菱形 / 选择框判命中，多个命中取最近的。

好处是原版命中规则一条不改，只换了「触点在哪」。测试脚本里的「点某个单位」也加了反方向的换算（原版脚底 / 头顶点 → 透视画面上身体中间的点），让自动化测试点在玩家真看见它的地方。

5. 踩过的坑

现象	以为是	实际	解法
远处一大片黑	被剔除了	把剔除范围放宽 4 倍、16 倍，黑的比例几乎不变（40.7% → 40.2%）。黑的是**天空**：俯视游戏没做天	俯仰限制，地平线不进画面
推上往回走	摇杆换算错	缺省朝向符号反了：原版镜头看向 −x −z，取成 +0.707，正好反 180°	缺省 / 重置朝向取 (−0.707, −0.707)
按住右推绕圈跑	镜头跟太快	摇杆相对镜头 + 镜头追摇杆 = 正反馈	参照镜头按一推锁住
改完整片地面墙体消失	—	一个计算着色器按正交视锥剔除，被一起换了矩阵	按着色器指纹排除，不改它
深度相关效果错位	—	着色器按「正交标志」选深度还原公式	标志置 0，按透视投影填参数；场景常量里的镜头位置、视线、线性深度起点同步改

6. 实测结果（最初实现，iOS App 跑在 Mac 上）

测什么	期望	结果
推摇杆上	朝镜头前方走	向前走 ✅
按住右推	直线跑，镜头转到身后	直线跑，约 1.5 秒转到背后 ✅
点箱子 / 点 NPC	命中并打开面板	命中开面板 ✅
技能钮朝首领拖动瞄准	锁中目标	锥内锁中，夹角 0° ✅
关掉开关	和不加这段代码一模一样	暂停页 / 统计页 0 像素差，0 行镜头日志 ✅

7. 现在的局限，适合接着做的方向

- **名字、血条、物品名仍按正交位置画**：这些界面元素由游戏的一个「世界坐标 → 屏幕坐标」投影函数算位置。根治 = 在这个函数出口换成透视坐标。
- **地下城缺墙、偏暗**：原版是按俯视设计的场景，透视下暴露出来；具体原因还没查。
- **点地面走路**没做（原版触摸操作本来就没有这个）。
- **第一人称**（`DM_CAM=2`）只有画面，输入不换算。
- **安卓 / 鸿蒙**的 Vulkan 后端没接：主相机签名和偏移是游戏数据，与后端无关，`cam_build` 的数学可以照搬，要接的是 Vulkan 侧绑定常量缓冲的那一处。

8. 想在别的游戏上复刻，按这个顺序

1. **找到注入点**：你要能在 GPU 读常量缓冲前改它（图形 API 翻译层、驱动钩子、RenderDoc 式拦截都行）。
2. **找主相机缓冲**：先扫所有常量缓冲，找带原版固定朝向的视图矩阵（等距游戏这个值是常数，最好认），记下布局：视图、投影、它们的逆、上一帧视图投影、深度还原参数。
3. **先只换矩阵**，镜头钉在人物身后，看画面立不立得起来；再逐个修：深度还原、剔除（找出按视锥剔除的那段，排除掉）、上一帧矩阵、场景常量里重复存的镜头位置。
4. **加跟随与俯仰限制**：墙钟阻尼、换区域重置、地平线不进画面。
5. **最后做输入反算**：摇杆按镜头朝向转、点目标按「目标那层深度」换回原版画面；参照镜头按一次操作锁住。
6. **做成开关**，缺省关，关着时和原版逐像素比一遍。

附录：核心代码节选

A. 主相机签名

```
static int cam_sig(const uint8_t *p, uint64_t size) {
    if (size < 608) return 0;
    const float *v = (const float *)(p + 80), *pr = (const float *)(p + 208);
    return fabsf(v[0] - 0.70711f) < 2e-3f && fabsf(v[1] + 0.35355f) < 2e-3f
        && fabsf(v[2] - 0.61237f) < 2e-3f && fabsf(v[5] - 0.86603f) < 2e-3f
        && fabsf(v[6] - 0.5f) < 2e-3f
        && pr[3] == 0 && pr[7] == 0 && pr[11] == 0 && fabsf(pr[15] - 1) < 1e-4f;
}
```

B. 人物位置与镜头跟随

```
// 原版逆视图：第 3 行 = 视线反方向 b，第 4 行 = 镜头位置 c；视线落地 = 人物
float t = by != 0 ? cy / by : 0, px = cx - t * bx, pz = cz - t * bz;
// 每帧：想要的朝向 = 瞄准 > 摇杆 > 位移方向；墙钟一阶阻尼
if (!have || 位移 >= 20) { fx = fz = -0.70711f; }        // 首帧 / 换区域：回到原版朝向
else if (want) {
    float a = atan2f(fz, fx), d = atan2f(wz, wx) - a;      // 规整到 (-π, π]
    a += d * (1 - exp(-dt / 0.35));  fx = cosf(a); fz = sinf(a);
}
```

C. 摇杆相对镜头

```
// 摇杆 → 画面方向 (x, y/2)
float sx = x / mag, sy = y / mag * 0.5f;  归一化(sx, sy);
// 画面方向 → 世界方向（f = 参照镜头水平朝向，右 = (-f.z, f.x)）
float wx = sx * -fz + sy * fx, wz = sx * fx + sy * fz;
// 世界方向 → 原版正交画面方向（人物脚下两点各投一次）→ 2:1 反换回摇杆
ox, oy = 原版画面方向;  kx = ox, ky = oy * 2;  归一化后乘回 mag
```

D. 点目标：搬到目标那一层

```
F = 目标脚底在世界中的位置（原版正交逆矩阵，连线交 y = 0）
射线 (o, d) = 触点在透视画面上的射线
n = 归一化(d.x, 0, d.z)                    // 竖直面法向
t = ((F − o) · n) / (d · n);  if (t <= 0) 跳过（在镜头背后）
Q = o + t·d;  原版画面点 = 原版正交视图投影(Q)  → 照原版命中框判
```